

Lesson 10 — Polynomial Evaluation

1. Where We Are Now

You have already completed the sparse matrix part of the course and started the polynomial part.

What you already learned before Lesson 10

Lesson	Main idea
Lesson 1	Why sparse matrices save memory
Lesson 2	Coordinate List representation and creation
Lesson 3	Complete C sparse matrix display program
Lesson 4	Sparse matrix addition algorithm
Lesson 5	Program to add two sparse matrices
Lesson 6	Complete sparse matrix addition in C
Lesson 7	Sparse matrix using C++ classes
Lesson 8	Sparse matrix operator overloading in C++
Lesson 9	Polynomial representation

Lesson 9 started the polynomial unit. In Lesson 9, you learned that a polynomial can be stored as an array of terms.

Each term stores:

coefficient, exponent

Lesson 10 now teaches how to **evaluate** that stored polynomial.

2. Main Goal of Lesson 10

The main goal of Lesson 10 is to calculate the value of a polynomial for a given value of x .

For example, suppose we have this polynomial:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

If we are told:

$$x = 2$$

Then we must replace every x with 2 and calculate the final answer.

So the polynomial becomes:

$$3(2^5) + 2(2^4) + 5(2^2) + 2(2) + 7$$

The final value is:

$$159$$

So:

$$P(2) = 159$$

3. What Does “Evaluate a Polynomial” Mean?

To **evaluate** a polynomial means:

Substitute a value for x and calculate the result.

Simple meaning:

Put a number into x , then solve the polynomial.

Example:

$$P(x) = 3x^2 + 2x + 1$$

If:

$$x = 4$$

Then:

$$P(4) = 3(4^2) + 2(4) + 1$$

Calculate:

$$\begin{aligned} 4^2 &= 16 \\ 3 \times 16 &= 48 \\ 2 \times 4 &= 8 \end{aligned}$$

So:

$$\begin{aligned} P(4) &= 48 + 8 + 1 \\ P(4) &= 57 \end{aligned}$$

That is polynomial evaluation.

4. Quick Reminder from Lesson 9

In Lesson 9, a polynomial was stored using two structures.

4.1 The `term` structure

One term stores one part of the polynomial.

```
struct term {  
    int coef;  
    int exp;  
};
```

Meaning of fields

Field	Meaning
<code>coef</code>	coefficient
<code>exp</code>	exponent

Example:

```
struct term t;  
t.coef = 3;  
t.exp = 5;
```

This represents:

$3x^5$

So one `term` stores:

```
coefficient = 3  
exponent = 5
```

5. The `poly` Structure

A polynomial has many terms, so we use another structure called `poly`.

```
struct poly {  
    int n;  
    struct term *t;  
};
```

Meaning of fields

Field	Meaning
<code>n</code>	number of terms in the polynomial
<code>t</code>	pointer to a dynamic array of terms

Example polynomial:

$3x^5 + 2x^4 + 5x^2 + 2x + 7$

This polynomial has 5 terms.

So:

$$n = 5$$

The dynamic array `t` stores the 5 terms.

6. Very Important Meaning of `n`

In sparse matrix lessons, `n` often meant:

number of columns

But in polynomial lessons, `n` means:

number of terms

This is very important.

Example:

$$3x^5 + 7$$

This polynomial has 2 terms:

$$\begin{array}{l} 3x^5 \\ 7 \end{array}$$

So:

$$n = 2$$

Even though the highest exponent is 5, `n` is not 5.

Correct:

`n` = number of terms

Wrong:

n = highest exponent

7. How a Polynomial Is Stored

Take this polynomial:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

It is stored as coefficient/exponent pairs.

Index	Coefficient	Exponent	Term
0	3	5	$3x^5$
1	2	4	$2x^4$
2	5	2	$5x^2$
3	2	1	$2x$
4	7	0	7

In memory, the array looks like this:

```
t[0] = {3, 5}
t[1] = {2, 4}
t[2] = {5, 2}
t[3] = {2, 1}
t[4] = {7, 0}
```

Each item means:

{coefficient, exponent}

8. Main Formula for Polynomial Evaluation

Each term is evaluated using this formula:

```
term value = coefficient * x^exponent
```

In code:

```
p.t[i].coef * pow(x, p.t[i].exp)
```

Meaning:

```
p.t[i].coef = coefficient of the current term  
p.t[i].exp  = exponent of the current term  
x           = value given by the user  
pow(x, exponent) = x raised to that exponent
```

9. Simple Term Example

Suppose the term is:

```
3x^5
```

And:

```
x = 2
```

Then:

```
3x^5 = 3 * 2^5
```

Calculate the power first:

```
2^5 = 32
```

Then multiply by the coefficient:

```
3 * 32 = 96
```

So:

$$3x^5 = 96 \text{ when } x = 2$$

10. Why We Use `pow()`

In C and C++, the function `pow()` is used to calculate powers.

Example:

```
pow(2, 5)
```

This means:

```
2^5
```

Result:

```
32
```

So if we write:

```
pow(x, p.t[i].exp)
```

It means:

```
x raised to the exponent of the current term
```

11. Header File for `pow()`

To use `pow()` in C++, include:

```
#include <cmath>
```

To use `pow()` in C, include:

```
#include <math.h>
```

In this lesson, the C++ version uses:

```
#include <cmath>
```

12. Why the Evaluation Function Returns `double`

The `pow()` function returns a floating-point value.

That is why the evaluation function is written like this:

```
double evaluate(struct poly p, int x)
```

The return type is:

```
double
```

This allows the function to return decimal or floating-point results if needed.

Even if the answer looks like an integer, using `double` is safe here because `pow()` works with floating-point numbers.

13. The Running Sum Idea

Polynomial evaluation uses a variable called `sum`.

This variable stores the total value as we evaluate each term.

At the beginning:

```
double sum = 0.0;
```

Why start with zero?

Because before evaluating any term, the total is zero.

Then for each term, we calculate:

```
coefficient * x^exponent
```

and add it to `sum`.

The idea is:

```
sum = sum + current term value
```

In C++ shorthand:

```
sum += current term value;
```

14. Evaluation Algorithm in Simple Steps

The algorithm is:

1. Start `sum = 0`
2. Visit the first term
3. Calculate `coefficient * x^exponent`
4. Add the term value to `sum`
5. Move to the next term
6. Repeat until all terms are finished
7. Return `sum`

This is a very direct algorithm because every term is processed once.

15. Evaluation Function

The main function of Lesson 10 is:

```
double evaluate(struct poly p, int x) {  
    double sum = 0.0;  
  
    for (int i = 0; i < p.n; i++) {  
        sum += p.t[i].coef * pow(x, p.t[i].exp);  
    }  
}
```

```
    return sum;
}
```

16. Line-by-Line Explanation of `evaluate()`

Function header

```
double evaluate(struct poly p, int x)
```

This means:

Part	Meaning
<code>double</code>	the function returns a floating-point value
<code>evaluate</code>	function name
<code>struct poly p</code>	polynomial to evaluate
<code>int x</code>	value of <code>x</code>

Initialize sum

```
double sum = 0.0;
```

This creates a variable called `sum`.

It starts at zero.

This variable will store the final answer step by step.

Loop through all terms

```
for (int i = 0; i < p.n; i++)
```

This loop starts from the first term and goes to the last term.

If:

```
p.n = 5
```

Then the loop runs for:

```
i = 0, 1, 2, 3, 4
```

That means it visits all 5 terms.

Evaluate one term and add it

```
sum += p.t[i].coef * pow(x, p.t[i].exp);
```

This line means:

1. Get the coefficient of the current term.
2. Get the exponent of the current term.
3. Calculate x raised to that exponent.
4. Multiply by the coefficient.
5. Add the result to sum.

Expanded version:

```
sum = sum + p.t[i].coef * pow(x, p.t[i].exp);
```

Return the answer

```
return sum;
```

After all terms are processed, `sum` contains the value of the polynomial.

So the function returns `sum`.

17. Why We Use `.` Instead of `->` Inside `evaluate()`

The function header is:

```
double evaluate(struct poly p, int x)
```

Here, `p` is a normal structure variable, not a pointer.

So we use the dot operator:

```
p.n  
p.t[i].coef  
p.t[i].exp
```

We do not use:

```
p->n  
p->t[i].coef  
p->t[i].exp
```

Simple rule:

Situation	Operator
Normal structure variable	<code>.</code>
Pointer to structure	<code>-></code>

Since `p` is not a pointer inside `evaluate()`, use `.`.

18. Full Dry Run

Let the polynomial be:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

Stored as:

i	Coefficient	Exponent	Term
0	3	5	<code>3x^5</code>
1	2	4	<code>2x^4</code>
2	5	2	<code>5x^2</code>

i	Coefficient	Exponent	Term
3	2	1	2x
4	7	0	7

Let:

$$x = 2$$

Initial value:

$$\text{sum} = 0$$

Step 1: Evaluate $3x^5$

Current term:

$$3x^5$$

Coefficient:

$$3$$

Exponent:

$$5$$

Calculate:

$$3 \times 2^5$$

First:

$$2^5 = 32$$

Then:

$$3 \times 32 = 96$$

Update sum:

$$\begin{aligned}\text{sum} &= 0 + 96 \\ \text{sum} &= 96\end{aligned}$$

Step 2: Evaluate $2x^4$

Current term:

$$2x^4$$

Calculate:

$$2 \times 2^4$$

First:

$$2^4 = 16$$

Then:

$$2 \times 16 = 32$$

Update sum:

$$\begin{aligned}\text{sum} &= 96 + 32 \\ \text{sum} &= 128\end{aligned}$$

Step 3: Evaluate $5x^2$

Current term:

$$5x^2$$

Calculate:

$$5 \times 2^2$$

First:

$$2^2 = 4$$

Then:

$$5 \times 4 = 20$$

Update sum:

$$\begin{aligned} \text{sum} &= 128 + 20 \\ \text{sum} &= 148 \end{aligned}$$

Step 4: Evaluate 2x

Current term:

$$2x$$

Remember:

$$2x = 2x^1$$

Calculate:

$$2 \times 2^1$$

First:

$$2^1 = 2$$

Then:

$$2 \times 2 = 4$$

Update sum:

$$\begin{aligned} \text{sum} &= 148 + 4 \\ \text{sum} &= 152 \end{aligned}$$

Step 5: Evaluate 7

Current term:

$$7$$

Remember:

$$7 = 7x^0$$

Because:

$$x^0 = 1$$

Calculate:

$$7 \times 2^0$$

First:

$$2^0 = 1$$

Then:

$$7 \times 1 = 7$$

Update sum:

```
sum = 152 + 7  
sum = 159
```

19. Final Answer of the Dry Run

The final value is:

159

So:

$P(2) = 159$

This means the polynomial value becomes 159 when $x = 2$.

20. Full Beginner-Friendly C++ Program

```
#include <iostream>  
#include <cmath>  
using namespace std;  
  
struct term {  
    int coef;  
    int exp;  
};  
  
struct poly {  
    int n;  
    struct term *t;  
};  
  
void create(struct poly *p) {  
    cout << "Enter number of terms: ";  
    cin >> p->n;
```

```

    p->t = new term[p->n];

    cout << "Enter coefficient and exponent for each term:\n";
    for (int i = 0; i < p->n; i++) {
        cin >> p->t[i].coef >> p->t[i].exp;
    }
}

void display(struct poly p) {
    for (int i = 0; i < p.n; i++) {
        cout << p.t[i].coef << "x^" << p.t[i].exp;

        if (i != p.n - 1) {
            cout << " + ";
        }
    }
    cout << endl;
}

double evaluate(struct poly p, int x) {
    double sum = 0.0;

    for (int i = 0; i < p.n; i++) {
        sum += p.t[i].coef * pow(x, p.t[i].exp);
    }

    return sum;
}

int main() {
    struct poly p;
    int x;

    create(&p);

    cout << "Polynomial is:\n";
    display(p);

    cout << "Enter value of x: ";
    cin >> x;

    cout << "Polynomial value is: " << evaluate(p, x) << endl;

    delete[] p.t;

    return 0;
}

```

21. Explanation of the Full Program

Header files

```
#include <iostream>
#include <cmath>
```

`iostream` is used for:

```
cin
cout
```

`cmath` is used for:

```
pow()
```

Namespace

```
using namespace std;
```

This allows us to write:

```
cout
cin
endl
```

instead of:

```
std::cout
std::cin
std::endl
```

term structure

```
struct term {  
    int coef;  
    int exp;  
};
```

This stores one polynomial term.

Example:

3x⁵

is stored as:

```
coef = 3  
exp = 5
```

poly structure

```
struct poly {  
    int n;  
    struct term *t;  
};
```

This stores the whole polynomial.

It has:

```
n = number of terms  
t = dynamic array of terms
```

create() function

```
void create(struct poly *p)
```

This function receives a pointer because it must modify the original polynomial.

It reads:

```
number of terms  
coefficient and exponent of each term
```

It also creates the dynamic array:

```
p->t = new term[p->n];
```

Because `p` is a pointer inside `create()`, we use:

```
p->n  
p->t
```

`display()` **function**

```
void display(struct poly p)
```

This function prints the polynomial.

It receives the polynomial by value, so we use:

```
p.n  
p.t[i].coef  
p.t[i].exp
```

The output is simple and explicit:

```
3x^5 + 2x^4 + 5x^2 + 2x^1 + 7x^0
```

This is mathematically the same as:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

`evaluate()` function

```
double evaluate(struct poly p, int x)
```

This function calculates the value of the polynomial for the given `x`.

It uses:

```
sum += p.t[i].coef * pow(x, p.t[i].exp);
```

This is the most important line in Lesson 10.

`main()` function

```
int main() {  
    struct poly p;  
    int x;  
  
    create(&p);  
  
    cout << "Polynomial is:\n";  
    display(p);  
  
    cout << "Enter value of x: ";  
    cin >> x;  
  
    cout << "Polynomial value is: " << evaluate(p, x) << endl;  
  
    delete[] p.t;  
  
    return 0;  
}
```

The `main()` function does these steps:

1. Creates a polynomial variable `p`.
2. Creates an integer variable `x`.
3. Calls `create(&p)` to read the polynomial.
4. Displays the polynomial.
5. Reads the value of `x`.
6. Calls `evaluate(p, x)`.

7. Prints the answer.
8. Releases dynamic memory using `delete[]`.

22. Example Input

```
Enter number of terms: 5
Enter coefficient and exponent for each term:
3 5
2 4
5 2
2 1
7 0
Enter value of x: 2
```

This represents:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

with:

```
x = 2
```

23. Example Output

```
Polynomial is:
3x^5 + 2x^4 + 5x^2 + 2x^1 + 7x^0
Polynomial value is: 159
```

The output prints `2x^1` instead of `2x`, and `7x^0` instead of `7`.

That is okay for beginner code because it shows every coefficient and exponent clearly.

24. Memory Management

The polynomial object is created like this:

```
struct poly p;
```

This creates the main polynomial structure.

The term array is created dynamically:

```
p->t = new term[p->n];
```

Because the array is created using `new[]`, it must be released using:

```
delete[] p.t;
```

This prevents memory leaks.

Memory rule:

Allocation	Deallocation
<code>new</code>	<code>delete</code>
<code>new[]</code>	<code>delete[]</code>
<code>malloc</code>	<code>free</code>

In this lesson:

```
new[] goes with delete[]
```

25. Time Complexity

The `evaluate()` function has this loop:

```
for (int i = 0; i < p.n; i++)
```

This loop visits every term once.

If the polynomial has `n` terms, the loop runs `n` times.

So the time complexity is:

$O(n)$

Meaning:

The running time grows based on the number of terms.

Example:

Number of terms	Number of loop iterations
3	3
5	5
10	10
100	100

So evaluation is linear in the number of terms.

26. Space Complexity

The evaluation function mainly uses one extra variable:

```
double sum;
```

It does not create another array.

It does not store all intermediate term values separately.

So the extra space complexity is:

$O(1)$

Meaning:

The extra memory used by evaluation stays constant.

Important distinction:

The polynomial itself uses $O(n)$ space to store terms.
The `evaluate()` function uses $O(1)$ extra space while calculating.

27. Common Beginner Mistakes

Mistake 1: Thinking `n` means highest exponent

Wrong:

`n = highest exponent`

Correct:

`n = number of terms`

Example:

`3x^5 + 7`

This has 2 terms, so:

`n = 2`

not:

`n = 5`

Mistake 2: Forgetting to initialize `sum`

Wrong:

```
double sum;
```

Correct:

```
double sum = 0.0;
```

If `sum` is not initialized, it may contain garbage value.

Then the final answer may be wrong.

Mistake 3: Using `->` when `p` is not a pointer

Inside `evaluate()`, the parameter is:

```
struct poly p
```

So `p` is a normal structure variable.

Correct:

```
p.n  
p.t[i].coef  
p.t[i].exp
```

Wrong:

```
p->n  
p->t[i].coef  
p->t[i].exp
```

Use `->` only when you have a pointer.

Mistake 4: Forgetting the constant term rule

A constant like:

```
7
```

is stored as:

```
7x^0
```

because:

$$x^0 = 1$$

So when evaluating at `x = 2`:

$$7x^0 = 7 \times 2^0 = 7 \times 1 = 7$$

Mistake 5: Forgetting to include `<cmath>`

If you use `pow()`, you need:

```
#include <cmath>
```

Without this header file, the compiler may give an error.

Mistake 6: Forgetting to release memory

If the term array is created using:

```
new term[p->n]
```

then release it using:

```
delete[] p.t;
```

This should be done when the polynomial is no longer needed.

28. Important Vocabulary

Term	Simple meaning
Polynomial	An expression made by adding terms
Term	One part of a polynomial
Coefficient	The number multiplied by the variable part

Term	Simple meaning
Exponent	The power of <code>x</code>
Evaluate	Substitute a value for <code>x</code> and calculate the result
<code>pow()</code>	Function used to calculate powers
Running sum	A variable that stores the total while adding values
<code>n</code>	Number of terms in the polynomial
<code>t</code>	Dynamic array of terms
<code>coef</code>	Coefficient field of a term
<code>exp</code>	Exponent field of a term

29. Key Code to Remember

The most important line in Lesson 10 is:

```
sum += p.t[i].coef * pow(x, p.t[i].exp);
```

Simple meaning:

```
Evaluate the current term and add it to the total.
```

Expanded meaning:

```
sum = sum + coefficient × xexponent
```

30. Lesson 10 Summary

Lesson 10 teaches how to evaluate a polynomial stored as an array of terms.

The key idea is:

```
For each term:
coefficient × xexponent
```

Then add all term values into one running total.

The evaluation function:

```
double evaluate(struct poly p, int x)
```

returns the value of the polynomial for the given `x`.

For the polynomial:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

when:

$$x = 2$$

The answer is:

$$159$$

So:

$$P(2) = 159$$

31. What You Should Be Able to Do After Lesson 10

After this lesson, you should be able to:

1. Explain what polynomial evaluation means.
2. Identify coefficient and exponent in each term.
3. Store a polynomial using term and poly structures.
4. Use `pow()` to calculate `x` raised to an exponent.
5. Use a running sum to add all term values.
6. Write an `evaluate()` function.
7. Dry run polynomial evaluation manually.
8. State the time complexity as $O(n)$.

9. State the extra space complexity as $O(1)$.
10. Avoid common mistakes with n , sum , $.$, $\rightarrow$, constants, and memory cleanup.

32. Quick Practice

Evaluate this polynomial:

$$2x^3 + 4x^2 + 5$$

when:

$$x = 3$$

Step by step:

$$\begin{aligned} 2x^3 &= 2 \times 3^3 = 2 \times 27 = 54 \\ 4x^2 &= 4 \times 3^2 = 4 \times 9 = 36 \\ 5 &= 5 \times 3^0 = 5 \times 1 = 5 \end{aligned}$$

Add:

$$54 + 36 + 5 = 95$$

So:

$$P(3) = 95$$

33. Connection to Lesson 11

Lesson 10 evaluates one polynomial.

Lesson 11 will use the same polynomial representation, but the goal will be different.

Instead of calculating the value of one polynomial, Lesson 11 will teach how to:

Add two polynomials by comparing their exponents.

This will be similar to sparse matrix addition because both use a merge-style idea.